

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

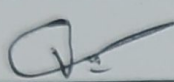
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

24

Course Code:	GSA1407	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	M. Vishnu Prasad	Reg. No.:	192311214
Section / Batch:	2023	Date:	14-07-2026

Code of Conduct

(192311214)
I, M. Vishnu Prasad (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	9 / 4	4 / 4	4 / 4	4 / 4	3 / 4	96.25
2	4 / 4	3 / 4	3 / 4	4 / 4	4 / 4	88.75
3	3 / 4	4 / 4	4 / 4	4 / 4	3 / 4	91.25
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	90.25
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. DA

a) Lexical analysis :-

$$P = (a * b) + (b * c) * 2$$

Lexeme

Token type

P

Identifier

=

Assignment operator

(

Left parenthesis

a

Identifier

*

Multiplication operator

b

Identifier

)

parenthesis

+

Addition operator

(

Parenthesis

b

Identifier

*

Mul. operator

c

identifier

)

parenthesis

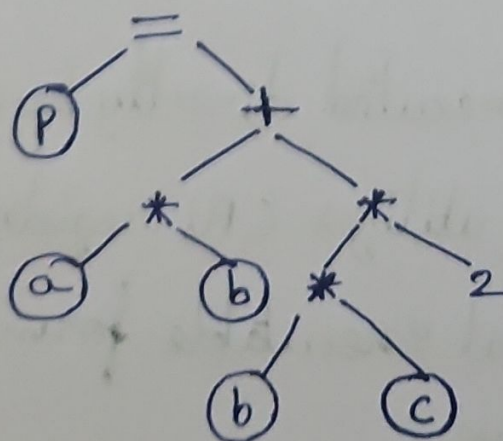
*

Mul. operator

2

Integer

b) Syntax tree :-



c) Intermediate code generation:-

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$t_4 = t_1 + t_3$

$p = t_4$

d) Code optimization:-

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 + t_2$

$p = t_1 + t_3$

e) Final target code:-

MOV R₁, a

MUL R₁, b

MOV R₂, b

MUL R₂, c

SHL R₂, 1

ADD R₁, R₂

MOV P, R₁

Significance:-

→ It is the machine dependent code produced by the compiler.

→ It can be executed directly by the processor.

→ It efficiently utilizes CPU registers & instructions.

→ It is the final executable form generated after all compiler phases.

a) $(\epsilon + aa^*)^*$

All strings consisting of zero or more a's

$$L = \{a^n \mid n \geq 0\}$$

• ϵ

• a

• aa

• aaa

• aaaa

b) $(a^* + b^*)abb$

Any combination of a's & b's followed by abb.

$$L = \{wabb \mid w \in \{a, b\}^*\}$$

• abb

• aabb

• babb

• aaabb

• bbabb

c) $(a^*b^*)^*aba(a^* + b^*)^*$

All strings over $\{a, b\}$ containing the substring "aba".

• aba

• aaba

• abab

• abaa

• babab

d) $(a+ab)^*$
strings formed by repeating either a or ab any number of.

- ϵ
- a
- ab
- aa
- aab

e) $(aba)^* + (a^*b^*)^*$

All possible strings over $\{a, b\}$, including the empty string.

- ϵ
- a
- b
- ab
- ba

Q3)

i)

S.No	Tokens
1	main
2	(
3	)
4	{
5	int
6	a
7	=
8	10
9	;

ii) Lexical errors :-

$$x = 1xab;$$

$$\text{Error} = 1xab$$

∴ Invalid numeric constant. Identifiers cannot start with a digit and $1xab$ is neither a valid integer nor a valid hexadecimal constant.

Q4) i) Languages of all strings having an even number of 0's

$$R.E = 1^*(01^*01^*)^*$$

ii) Language of all strings containing at least one 1

$$R.E = (0+1)^*1(0+1)^*$$

iii) Language of all strings in which both the number of 0's & the number of 1's are odd

$$R.E = (00+11)^*(01+10)(00+11)^*((01+10)(00+11)^*(01+10)(00+11)^*)^*$$

iv) Language of all strings that start with 1 & end with 0.

$$R.E = 1(0+1)^*0$$

v) Language of all strings that do not contain consecutive 1's.

$$R.E = (0+10)^*(\epsilon+1)$$

S.No	Token
1	int
2	var 123
3	=
4	12345
5	;
6	float
7	var 456
8	=
9	123.45
10	;
11	{
12	(
13	var 123
14	= =
15	var 456
16	)
17	{
18	return
19	;
20	}

∴ Total number of tokens = 20 //

b) Keywords:-

→ int
→ float
→ if
→ return

Operators:-

→ =
→ =
→ ==

Separators:-

→ ;
→ ;
→ (
→)
→ {
→ ;
→ }

Identifiers:-

→ Var123
→ Var456
→ Var123
→ Var456

Integer literals:-

→ 12345

Floating point literals:-

→ 123.45

c)

Token	Transition
int	3
var123	6
=	1
12345	5
;	1
float	5
var456	6
=	1
123.45	6
.	1

Token	Transition
123.45	6
;	1
if	2
(	1
Var123	6
==	2
Var456	6
)	1
{	1
return	6

Token	Transition
;	1
}	1
Total	70

∴ FSM transitions = 70